

ComfyChair

Documento de Decisiones de Diseño

Trabajo Práctico 1 — Técnicas y Herramientas de Software 2026
Mayo 2026

Participantes:

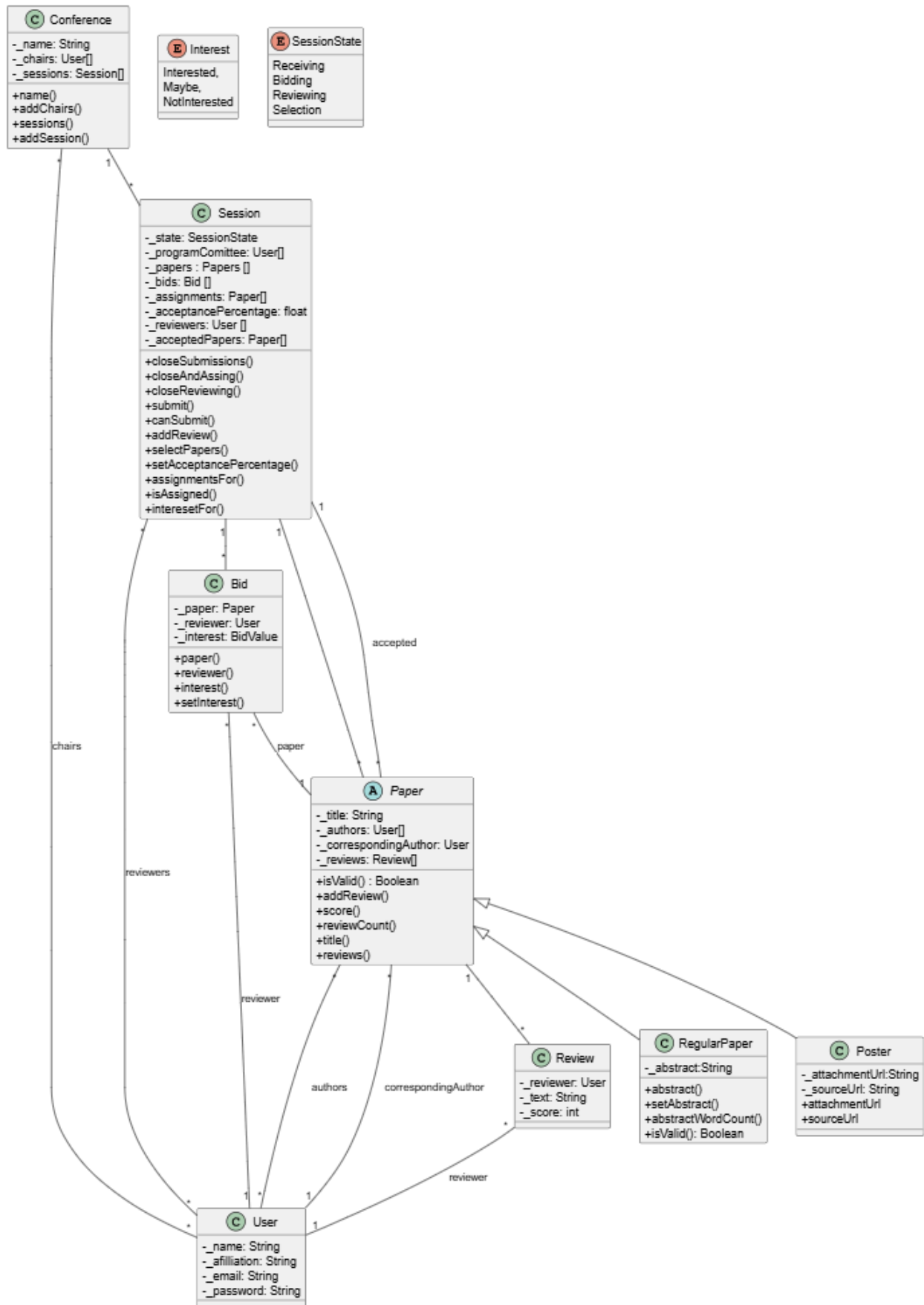
Mucci Mauro

Nuñez Leandro

El código base provisto cubría las etapas de Recepción y Bidding. La implementación propia abarca las secciones 4.1 (asignación de revisores), 4.2 (carga de revisiones) y 4.3 (selección de artículos) del enunciado.

Diagrama de clases

Para acceder a este documento, es necesario descargar la extensión Plant UML, luego, en el archivo .puml, presionar alt+D para observar la siguiente imagen:



Archivos modificados del código base

1. src/[Session.js](#)

- a. Por qué fue necesario modificarlo: Session.js es el único archivo del código base que fue modificado. Todos los archivos restantes del código base no fueron modificados.
- b. Qué se agregó y por qué
 - i. Nuevos atributos en el constructor:
 1. `_assignments`: Map, asocia cada Paper con la lista de revisores asignados.
 2. `_acceptancePercentage`: Number (default 100) — porcentaje máximo de aceptación para la selección
 - ii. Métodos de transición de etapa:
 1. `closeAndAssign()`: avanza de Bidding a Reviewing y ejecuta internamente `_assignReviewers()`.
 2. `closeReviewing()`: avanza de Reviewing a Selection.
 3. Ambigüedad resuelta: el enunciado describe una etapa 'Asignación y Revisión' como una sola, lo que podría interpretarse como dos sub-etapas. Se decidió implementarla como una única etapa 'Reviewing', donde la asignación ocurre instantáneamente al entrar.
 - iii. Método `_assignReviewers()`:
 1. Cálculo de capacidad: se calcula $\text{base} = \text{floor}(3A/R)$ y $\text{extra} = (3A) \bmod R$. Los primeros 'extra' revisores reciben capacidad $\text{base}+1$ y el resto base. Esto distribuye exactamente $3A$ revisiones entre R revisores de la forma más equitativa posible, tal como exige el enunciado.
 2. Algoritmo por rondas: se realizan 3 rondas; en cada ronda se asigna exactamente un revisor a cada artículo. Esto evita el problema del algoritmo greedy artículo por artículo, donde los primeros artículos consumen la mayor parte de la capacidad y los últimos no consiguen 3 revisores.
 3. Prioridad de bids: Interested (0) > Maybe (1) > Sin bid expresado (2) > NotInterested (3). Este orden respeta exactamente la sección 2.5 del enunciado.
 4. Conflicto de interés: se excluye a cualquier revisor que figure en `paper._authors`. Se accede al atributo privado directamente porque Paper no expone un método `authors()` público y el enunciado indica no modificar el código base salvo defectos justificados.
 5. Capacidad de emergencia: si todos los revisores elegibles tienen capacidad 0 por efecto del conflicto de interés, se presta capacidad extra al primer revisor disponible no-autor. Esto garantiza llegar a 3 revisores siempre que existan suficientes revisores no-autores.

6. Ambigüedad resuelta: el enunciado no especifica qué hacer cuando el conflicto de interés hace imposible respetar la distribución de carga original. Se decidió priorizar la restricción de integridad (3 revisores no-autores por artículo) por sobre la distribución exacta de carga. Si no existen suficientes revisores no-autores, se lanza un Error.
- iv. Métodos de consulta de asignaciones:
 1. `assignmentsFor(paper)` — retorna la lista de revisores asignados a un artículo.
 2. `isAssigned(paper, reviewer)` — retorna true si el revisor está asignado al artículo. Encapsula la lógica de verificación usada en `addReview()`.
 - v. e) Método `addReview(paper, reviewer, text, score)`
 1. Centraliza en `Session` todas las validaciones de negocio: (1) etapa correcta (`Reviewing`), (2) revisor asignado al artículo, (3) puntaje entero entre -3 y +3.
 2. Una vez validado, delega la creación del objeto `Review` a `paper.addReview()`.
 3. Ambigüedad resuelta: el enunciado establece que 'solo los revisores asignados pueden revisar' pero no especifica en qué clase debe validarse. Se decidió validarlo en `Session` porque es quien conoce las asignaciones, manteniendo `Paper` sin dependencia de `Session`.
 - vi. Métodos de selección:
 1. `setAcceptancePercentage(pct)` — configura el porcentaje antes de la selección. Valida que el valor esté entre 0 y 100.
 2. `selectPapers()` — ordena los artículos por score decreciente y retorna los primeros $\text{floor}(N * \text{pct} / 100)$. Solo puede invocarse en etapa `Selection`.
 3. Ambigüedades resueltas:
 - a. El enunciado no especifica el criterio de desempate cuando dos artículos tienen el mismo score en el límite del corte. Se optó por el orden de envío (orden original del `array_papers`), usando `Array.sort()`.
 - b. el enunciado dice 'hasta completar ese porcentaje' sin indicar cómo redondear. Se eligió `floor()` porque garantiza estrictamente no superar el límite máximo.

2. Carpeta tests

Se crearon tres nuevos archivos de tests, uno por cada sección de la consigna (4.1, 4.2 y 4.3).

1. tests/[Assignment.test.js](#)
 - a. 'should assign exactly 3 reviewers per paper': verifica el requisito fundamental del enunciado (sección 2.5: 'siempre debe haber exactamente 3 revisores asignados por artículo').
 - b. 'should transition to Reviewing stage after assignment': verifica que `closeAndAssign()` avanza la etapa correctamente para que el flujo continúe.

- c. 'should not allow closeAndAssign outside Bidding stage': verifica que la transición solo ocurre desde el estado correcto, evitando inconsistencias.
- d. 'should distribute 10 papers among 7 reviewers correctly (2 do 5, 5 do 4)': reproduce el ejemplo literal del enunciado (sección 2.5). Es el test más importante para verificar la fórmula de distribución.
- e. 'should distribute 3 papers among 3 reviewers (each gets 3)': verifica el caso base donde la división es exacta y no hay resto.
- f. 'should prefer Interested reviewers over Maybe': verifica que la prioridad de bids se respeta al asignar revisores.
- g. 'should fill remaining slots with Maybe if not enough Interested': verifica que los slots restantes se completan con el siguiente nivel de prioridad.
- h. 'should not assign a reviewer who is an author of the paper': verifica el conflicto de interés, requisito opcional del enunciado que fue implementado.

2. tests/[Review.test.js](#)

- a. 'should allow an assigned reviewer to submit a review': verifica el camino feliz: un revisor asignado puede cargar su revisión.
- b. 'should reject a review from a non-assigned reviewer': verifica la restricción del enunciado 'solo los revisores asignados al mismo pueden revisarlo'.
- c. 'should reject scores outside -3 to +3': verifica el rango de puntajes establecido en la sección 2.6 del enunciado.
- d. 'should accept boundary scores -3 and +3': verifica explícitamente que los valores límite del rango son válidos (inclusión de extremos).
- e. 'should reject reviews outside the Reviewing stage': verifica que no se pueden cargar revisiones en otras etapas.
- f. 'should not allow more than 3 reviews per paper': verifica el límite de revisiones por artículo establecido en la sección 2.6. Complementa el test equivalente de Paper.test.js desde la perspectiva del flujo completo.

3. tests/[Selection.test.js](#)

- a. 'should select top papers by score up to the acceptance percentage': verifica el comportamiento central: con 4 artículos y 50% de aceptación, se seleccionan los 2 de mayor score.
- b. 'should return no papers if percentage is 0': verifica el caso borde inferior (0% = ningún artículo aceptado).
- c. 'should return all papers if percentage is 100': verifica el caso borde superior (100% = todos los artículos aceptados), que también es el comportamiento por defecto.
- d. 'should not allow selection outside the Selection stage': verifica que selectPapers() solo puede invocarse en la etapa correcta.
- e. 'should reject invalid acceptance percentages': verifica que setAcceptancePercentage() lanza Error para valores negativos o mayores a 100.
- f. 'should use floor when percentage doesn't yield a whole number': verifica el comportamiento de redondeo (3 artículos al 34% = floor(1.02) = 1 aceptado), que fue una decisión de diseño explícita.

Uso de la IA

Para la realizacion de esta etapa se uso la IA Claude,

1. para favorecer el entendimiento del problema.
2. Para repasar la semantica de JS
3. Para optimizar el código
4. Para evaluar que la consigna este cubierta